

0.13. Une lettre toutes les secondes, avec attendre

Le temps dit directement en secondes : `attendre(1)`.

1. Ce qu'il faut comprendre

Ce qui est nouveau ici : `attendre(secondes)`. Elle fait partie de la console, comme `ALPHABET` : on s'en sert sans rien déclarer.

`attendre(1)` attend **une seconde**, puis le programme continue à la ligne suivante. À l'intérieur, elle compte les images à notre place : $1 \times 60 = 60$ images. On peut aller jusqu'à `attendre(255)`, plus de 4 minutes.

Le programme se lit dans l'ordre : **afficher** la lettre, **attendre** une seconde, passer à la **suivante**. Il n'y a plus de compteur `images`, ni d'`image()` : `attendre` s'en occupe.

La différence avec le 0.12 : pendant `attendre()`, le programme est **arrêté**. Rien d'autre ne se passe : la manette n'est pas lue. En comptant les images soi-même (0.12), la boucle de jeu continue de tourner pendant l'attente. Pour un jeu, c'est souvent ce qu'il faut ; pour faire patienter, `attendre` est plus simple.

La limite : `attendre` ne sait pas **ce qui** doit attendre : il fige **tout** le programme. Avec `attendre(10)`, aucune autre chose ne peut bouger pendant 10 secondes. Pour que plusieurs choses aient chacune leur rythme, voir le chapitre suivant.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <attendre>` → [0.12.3. La fonction attendre\(\) — attendre des secondes](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **893 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	52 o	← la console, à l'allumage — la mise en route de la console
main()	36 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← AttendreSecondes — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreSecondes	17 o	← attendre() — attend des secondes entières
CopierTuiles	18 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	704 o	← CopierTuiles — la police entière

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreAcces`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`,

EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
poser()    × 1    12 o sur place, dans main()
attendre() × 1    5 o sur place, dans main() + AttendreSecondes (17 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
#include <attendre>    // attend des secondes entières
```

2. UNE VARIABLE — LETTRE

```
uint8_t lettre = 0;  // l'indice de la lettre affichée
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    while (true) {
        poser(0, 0, ALPHABET[lettre]); // 1. affiche la lettre en haut à gauche
        attendre(1);                  // 2. attend 1 seconde : le nombre entre ( ) est en SECONDES
                                        // (tout le programme s'arrête pendant ce temps)

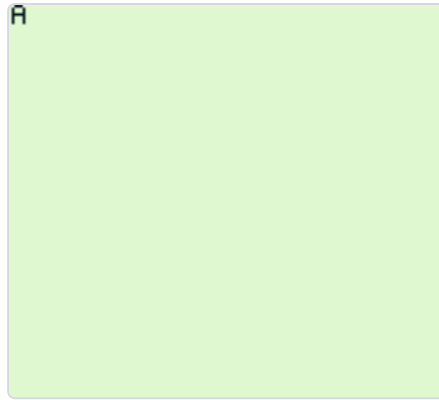
        lettre++;                      // 3. la lettre suivante
        if (lettre == sizeof(ALPHABET)) lettre = 0; // après Z, on revient à A
    }
}
```

3. Ce qui se passe, étape par étape

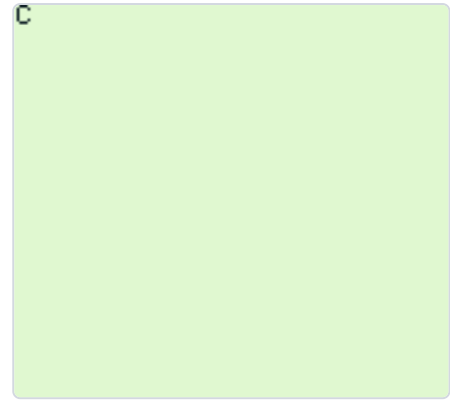
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



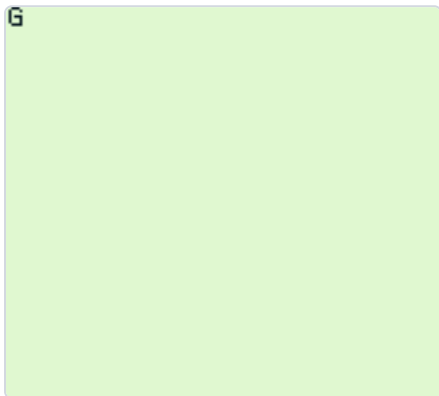
la toute première image — la console vient de démarrer — image 1



A
un tiers de seconde plus tard : le premier dessin est posé — image 20



C
au bout de trois secondes — image 180



G
au bout de sept secondes — image 420

4. Ce qu'on doit voir

En haut à gauche : A, puis B une seconde plus tard, puis C..., comme au 0.12.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ A au départ
- ✓ B une seconde plus tard
- ✓ C la seconde d'après

Ensuite : **0.13.1. Une lettre toutes les secondes, avec attendre — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++